Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование рядов Тейлора»

Выполнил:
студент группы 3825Б1ПМии1
Булганов Даниил Александрович

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя 4
Описание программной реализации 5
Результаты экспериментов 8
Заключение 10
Литература 11
Приложение 122

Введение

В вычислительной математике и численном анализе одной из ключевых задач является приближенное вычисление значений элементарных и специальных функций. Часто для этого используются разложения в степенные ряды, которые позволяют получить результат с любой наперед заданной точностью. Однако, при реализации таких вычислений на компьютере возникает проблема накопления вычислительной погрешности, связанная с ограниченной разрядной сеткой чисел с плавающей точкой.

Порядок суммирования членов ряда может существенно влиять на итоговую точность. Прямое суммирование (от первого члена к последнему) часто приводит к накоплению ошибок, особенно при наличии больших членов ряда с последующим взаимным уничтожением. Альтернативой является обратное суммирование, когда члены ряда складываются, начиная с наименьших, что позволяет уменьшить влияние ошибок округления.

Данная лабораторная работа направлена на практическое исследование влияния порядка суммирования на точность вычисления суммы ряда для четырех различных функций: sin(x), cos(x), exp(x) и ln(1+x). Результаты работы позволяют сделать выводы о преимуществах и недостатках каждого подхода.

Постановка задачи

Целью данной лабораторной работы является сравнить два способа суммирования членов ряда — от первого к последнему и от последнего к первому — и показать, что порядок суммирования влияет на результат: сумма, полученная при сложении «маленького с большим», отличается от суммы, полученной при сложении «большого с маленьким».

Для практического исследования были выбраны следующие функции: sin(x), cos(x), exp(x) и ln(1+x). Практическая часть заключалась в разработке программы на языке программирования C, способной корректно считать ряд Тейлора прямым и обратным суммированием, а так же считать погрешность от точного значения.

Разработанное консольное приложение предоставляет пользователю интерфейс для выбора одной из функций. После выполнения суммирования программа выводит на экран погрешность при прямом и обратном суммировании.

В рамках аналитической части работы было проведено экспериментальное исследование точности двух подходов к суммированию рядов. Путём построения графиков зависимости погрешности от количества членов ряда для каждой функции удалось наглядно продемонстрировать различие в накоплении вычислительной погрешности при прямом и обратном порядке суммирования.

Руководство пользователя

При запуске программы перед пользователем появляется меню (рис.1).

```
Выберите функцию:
1 - sin(x)
2 - cos(x)
3 - exp(x)
4 - ln(1+x)
Ваш выбор:
```

(рис.1)

Далее пользователь выбирает требуемую функцию. Для этого нужно выбрать цифру от 1 до 4, где 1 – вычисление sin(x), 2 – вычисление cos(x), 3 – вычисление exp(x), 4 – вычисление ln(1+x). После выбора функции программа начинает вычисление.

Программа выводит на экран (рис.2):

- Название выбранной функции
- Таблицу, где для каждого количества членов n (от 1 до 30) выводятся:
  - Абсолютная погрешность для прямой суммы
  - Абсолютная погрешность для обратной суммы

```
Функция: sin(x)
n=1 прямая погрешность=1,054402e+01 обратная погрешность=1,054402e+01
n=2 прямая погрешность=1,561227e+02 обратная погрешность=1,561227e+02
n=3 прямая погрешность=6,772107e+02 обратная погрешность=6,772107e+02
n=4 прямая погрешность=1,306916e+03 обратная погрешность=1,306916e+03
n=5 прямая погрешность=1,448816e+03 обратная погрешность=1,448816e+03
n=6 прямая погрешность=1,056395e+03 обратная погрешность=1,056395e+03
n=7 прямая погрешность=5,495091e+02 обратная погрешность=5,495092e+02
n=8 прямая погрешность=2,152073e+02 обратная погрешность=2,152071e+02
n=9 прямая погрешность=6,593849e+01 обратная погрешность=6,593863e+01
n=10 прямая погрешность=1,626787e+01 обратная погрешность=1,626779e+01
n=11 прямая погрешность=3,305074e+00 обратная погрешность=3,305084e+00
n=12 прямая погрешность=5,630963e-01 обратная погрешность=5,630803e-01
n=13 прямая погрешность=8,159882e-02 обратная погрешность=8,169508e-02
n=14 прямая погрешность=1,023811e-02 обратная погрешность=1,010180e-02
n=15 прямая погрешность=1,071870e-03 обратная погрешность=1,128674e-03
n=16 прямая погрешность=1,442432e-04 обратная погрешность=9,202957e-05
n=17 прямая погрешность=2,908707e-05 обратная погрешность=3,004074e-05
n=18 прямая погрешность=3,874302e-05 обратная погрешность=3,004074e-05
n=19 прямая погрешность=3,802776e-05 обратная погрешность=3,004074e-05
n=20 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=21 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=22 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=23 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=24 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=25 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=26 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=27 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=28 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=29 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
n=30 прямая погрешность=3,808737e-05 обратная погрешность=3,004074e-05
```

(рис.2)

Описание программной реализации

`float sin_term(float current, float x, int n)` Функция вычисляет следующий член ряда Тейлора для функции sin(x) на основе текущего члена. Механизм основан на рекуррентном соотношении: каждый последующий член ряда получается из предыдущего умножением на множитель $(-x^2)/((2n+2)(2n+3))$. Это позволяет избежать прямого вычисления факториалов и степеней, повышая эффективность вычислений. Функция принимает текущий член ряда current, аргумент x и номер текущего члена n, возвращая следующий член ряда.

`float cos_term(float current, float x, int n)` Функция вычисляет следующий член ряда Тейлора для функции cos(x) с использованием рекуррентного соотношения $(-x^2)/((2n+1)(2n+2))$. Аналогично функции `sin_term`, данная функция позволяет последовательно вычислять все члены ряда без пересчета факториалов. Принимает текущий член current, аргумент x и номер члена n, возвращает значение следующего члена ряда.

`float exp_term(float current, float x, int n)` Функция вычисляет следующий член ряда Тейлора для экспоненциальной функции exp(x) по рекуррентной формуле x/(n+1). Ряд exp(x) сходится при всех действительных x, что делает его удобным для демонстрации методов суммирования. Функция принимает текущий член ряда `current`, аргумент x и номер текущего члена n, возвращая следующий член ряда.

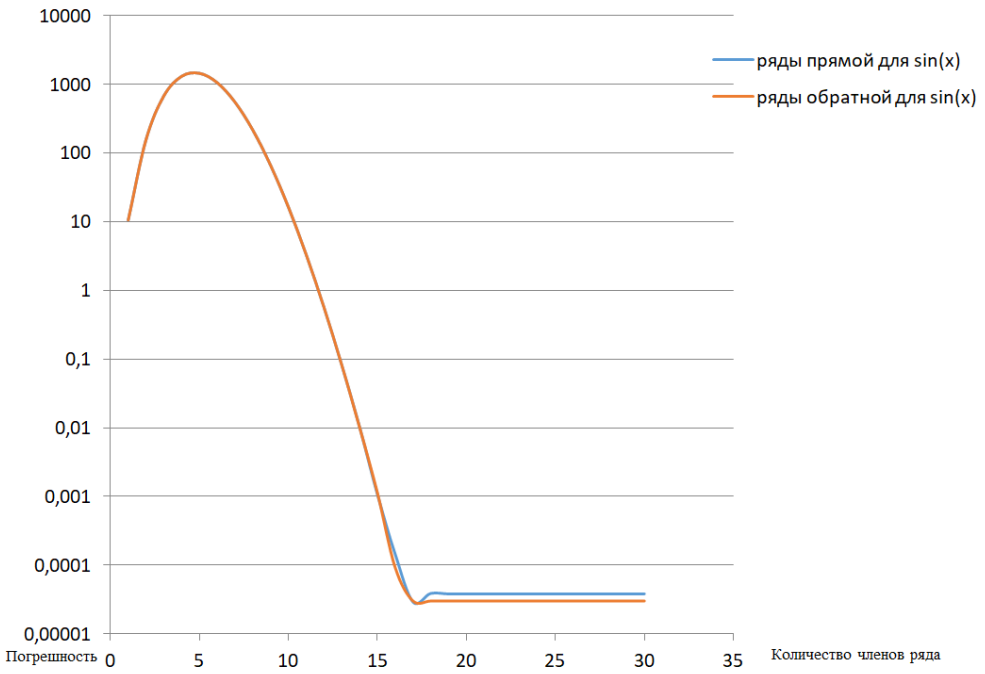
`float ln_term(float current, float x, int n)` Функция вычисляет следующий член ряда Тейлора для функции ln(1+x) с использованием рекуррентного соотношения. Особенность реализации заключается в различном вычислении для начального члена (n=0) и последующих: для n=0 используется множитель -x/2, для n>0 — множитель $(-x)\cdot n/(n+1)$. Функция принимает текущий член `current,` аргумент x и номер члена n, возвращая следующий член ряда.

`float pramayasum(float (*term_func)(float, float, int), float x, float first_term, int n)` Функция вычисляет сумму первых n членов ряда в прямом порядке — от первого к последнему. Она принимает указатель на функцию, которая вычисляет следующий член ряда, значение x, первый член и количество членов n. Сначала сумма и текущий член приравниваются к первому члену. Затем в цикле от 1 до n-1 вычисляется следующий член и прибавляется к сумме. Функция возвращает полученную сумму.

`float obratnayasum(float (*term_func)(float, float, int), float x, float first_term, int n)` Функция вычисляет сумму первых n членов ряда в обратном порядке — от последнего к первому. Она принимает те же параметры, что и прямая сумма. Сначала функция выделяет память под массив для хранения всех членов ряда. Затем в цикле вычисляются все члены от первого до последнего и сохраняются в массиве. После этого сумма вычисляется в обратном порядке: начиная с последнего члена и заканчивая первым. В конце выделенная память освобождается, и функция возвращает сумму. Обратный порядок суммирования позволяет уменьшить погрешность вычислений, так как сначала складываются самые маленькие числа.

`int main()` Главная функция управляет работой всей программы. Сначала она устанавливает русскую локализацию для корректного отображения сообщений. Затем выводит меню с четырьмя вариантами функций: sin(x), cos(x), exp(x) и ln(1+x), после чего считывает выбор пользователя с помощью `scanf_s`. В зависимости от выбранного номера в операторе `switch` задаются параметры для дальнейших вычислений: значение аргумента x, указатель на соответствующую функцию для вычисления членов ряда `term_func`, первый член ряда `first_term`, точное значение функции `exact_value`, вычисленное с помощью стандартных математических функций sin, cos, exp или ln, а также название функции `func_name` для вывода. Если пользователь вводит номер, не соответствующий ни одному из вариантов, программа выводит сообщение об ошибке и завершает работу с кодом возврата 1. После настройки параметров задается максимальное количество членов ряда `n_max` равное 30, и выводится название выбранной функции. Далее запускается цикл от n равного 1 до 30, в котором на каждом шаге вычисляются приближенные значения функции с помощью прямого суммирования через функцию `pramayasum` и обратного суммирования через функцию `obratnayasum`. Затем вычисляются абсолютные погрешности для обоих методов как модуль разности между приближенным и точным значением с помощью функции `fabsf`. Формулы вычисления `fabsf(pramaya - exact_value)` и `fabsf(obratnaya - exact_value).` После этого на экран выводится строка с номером n, погрешностью прямого суммирования и погрешностью обратного суммирования в экспоненциальном формате. По завершении цикла функция возвращает 0, что означает успешное выполнение программы.

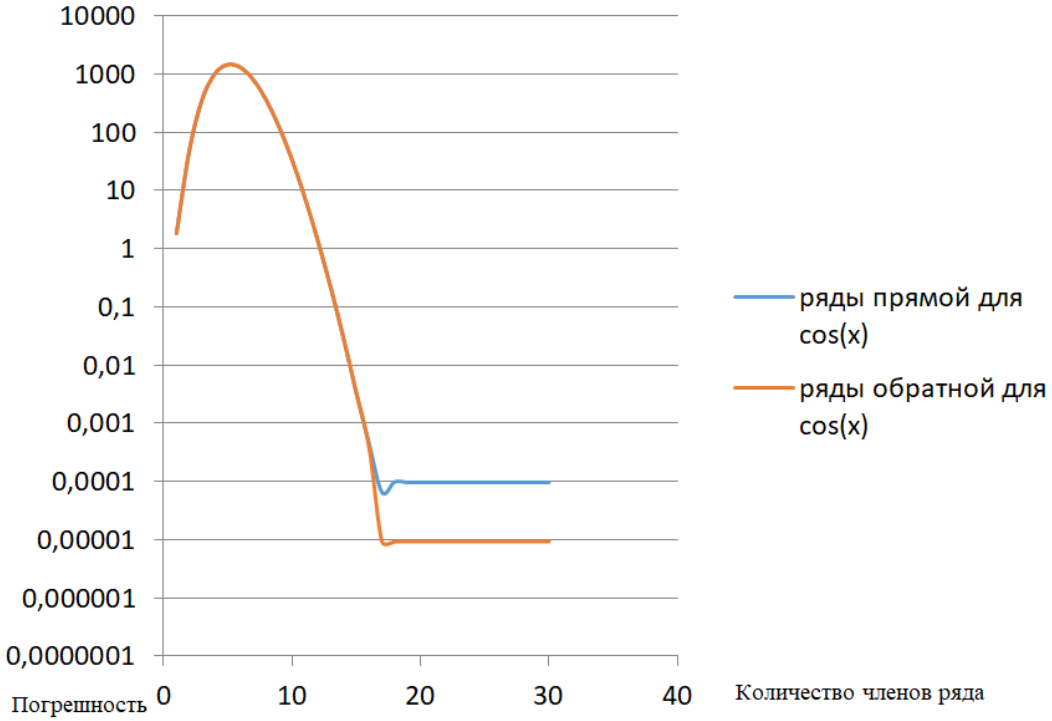
Результаты экспериментов



(рис.3)

Рис.3 График погрешности от количества членов ряда для синуса

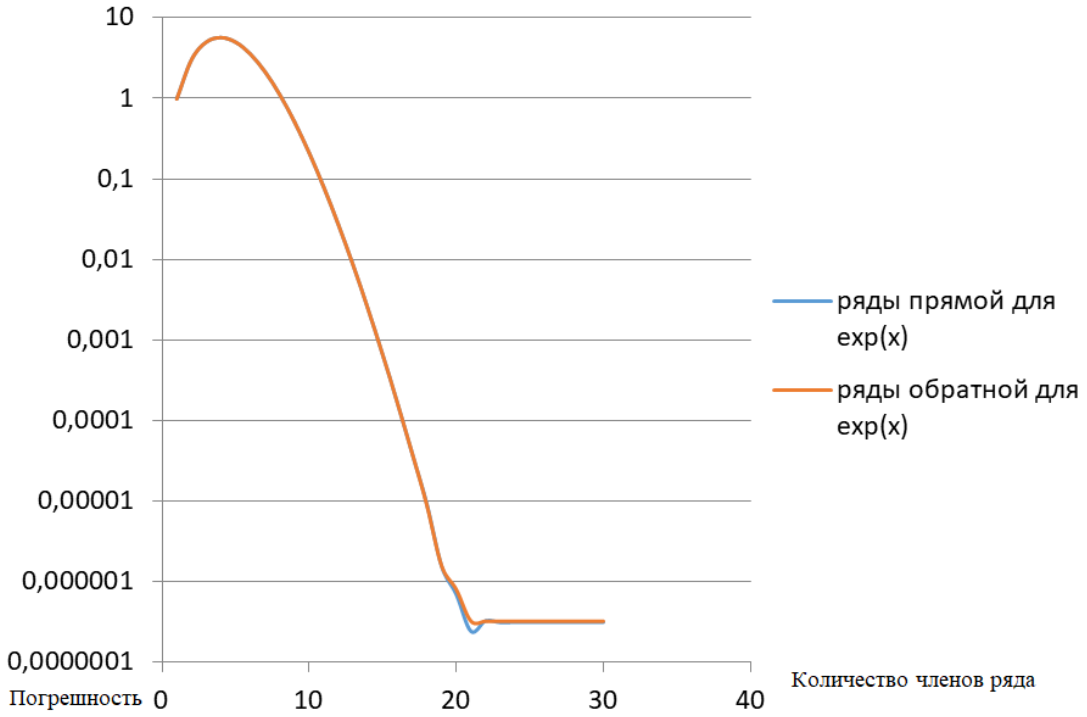
Из графика видно, что вначале графики идентичные, но после 15 члена начинаются небольшие различия. Погрешность для обратной суммы становится меньше.



(рис.4)

Рис.4 График погрешности от количества членов ряда для косинуса

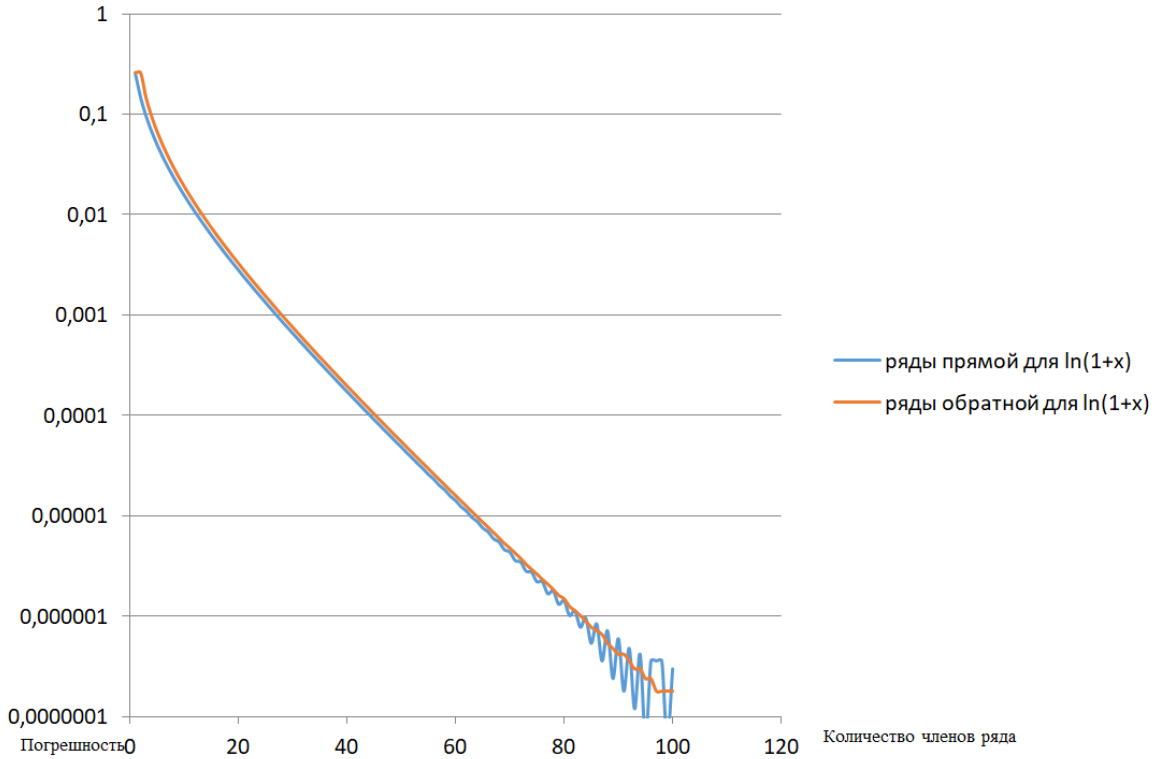
Из графика видно, что вначале графики идентичные, но примерно после 15 члена начинаются большие отличия. Погрешность для обратной суммы становится сильно меньше.



(рис.5)

Рис.5 График погрешности от количества членов ряда для экспоненты

Из графика видно, что вначале графики идентичные, но ближе к 20 члену начинаются небольшие отличия, которые заканчиваются примерно после 22 члена. Далее погрешность так же, как и вначале идентичная.



(рис.6)

Рис.6 График погрешности от количества членов ряда для натурального логарифма

Из графика видно, что с самого начала графики немного отличаются, но сильные различия начинаются ближе к 80 члену, которые далее только увеличиваются. Из графика видно, что для обратной суммы погрешность более стабильная.

Заключение

В ходе выполнения лабораторной работы были реализованы два способа суммирования членов ряда Тейлора для функций sin(x), cos(x), exp(x) и ln(1+x): прямой (от первого члена к последнему) и обратный (от последнего к первому). Экспериментально подтверждено, что порядок суммирования влияет на итоговую точность вычислений. При обратном суммировании, когда сложение начинается с наименьших по модулю слагаемых, накопление погрешности округления оказывается минимальным. Прямое суммирование, напротив, приводит к потере значимости младших разрядов из-за добавления малых величин к уже накопленной большой сумме. Построенные графики зависимости погрешности от количества членов ряда наглядно демонстрируют, что различие между двумя способами есть.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение

```
int main() {
    setlocale(LC_ALL, "Russian");
    int choice;
    printf("Выберите функцию:\n");
    printf("1 — sin(x)\n");
    printf("2 — cos(x)\n");
    printf("3 — exp(x)\n");
    printf("4 — ln(1+x)\n");
    printf("Ваш выбор: ");
    scanf_s("%d", &choice);

    float x, exact_value, first_term;
    float (*term_func)(float, float, int);
    const char* func_name;

    switch (choice) {
    case 1:
        x = 10.0f;
        term_func = sin_term;
        first_term = x;
        exact_value = sinf(x);
        func_name = "sin(x)";
        break;
    case 2:
        x = 10.0f;
        term_func = cos_term;
        first_term = 1.0f;
        exact_value = cosf(x);
        func_name = "cos(x)";
        break;
    case 3:
        x = -4.0f;
        term_func = exp_term;
        first_term = 1.0f;
        exact_value = expf(x);
        func_name = "exp(x)";
        break;
    case 4:
        x = 0.9f;
        term_func = ln_term;
        first_term = x;
        exact_value = logf(1.0f + x);
        func_name = "ln(1+x)";
        break;
    default:
        printf("Неверный выбор!\n");
        return 1;
    }

    int n_max = 30;
    printf("\nФункция: %s\n", func_name);

    for (int n = 1; n <= n_max; n++) {
        float pramaya = pramayasum(term_func, x, first_term, n);
        float obratnaya = obratnayasum(term_func, x, first_term, n);

        float pramaya_error = fabsf(pramaya - exact_value);
        float obratnaya_error = fabsf(obratnaya - exact_value);

        printf("n=%d прямая погрешность=%e обратная погрешность=%e\n", n,
pramaya_error, obratnaya_error);
    }
    return 0;
```